

---

# Benchmarking Robotic Manipulation under Physically Realistic Dynamics

---

Zhiyuan Li<sup>1,2</sup>, Ruiheng Yang<sup>1</sup>, Xuan Zhao<sup>1</sup>, and Amir Rasouli<sup>1,\*</sup>

<sup>1</sup>Huawei Technologies Canada      <sup>2</sup>University of Toronto

\*Email: amir.rasouli@huawei.com

## Abstract

Object manipulation in the dynamic real world requires understanding of how the environment evolves and how to respond to it accordingly. Existing benchmarks, however, do not adequately address these needs. Physics-based benchmarks focus primarily on the predictive capability of models while omitting the action component. Dynamic robotic manipulation benchmarks contain only basic, physically implausible object motions and evaluate policies synchronously, so physical anticipation is never necessary and inference cost is free. There remains a gap in evaluating both the predictive reasoning and the actions of robotic policies in physically realistic dynamic environments. To this end, we propose **RoboDyna**, a benchmark for evaluating the physical understanding and actions of manipulation policies in dynamic scenes. **RoboDyna** consists of 20 conceptual dynamic tasks with simplified objects and minimal scene clutter in five categories under a two-factor difficulty design, for evaluating the core physical understanding and capabilities of policies. In addition, 10 complex household tasks with realistic scenes are designed to measure how these skills transfer to real-world scenarios. Our benchmark contains 3,500 expert demonstrations and features protocols for synchronized (world frozen during inference) and asynchronous evaluation, success and partial metrics to measure progress, and interactive tools for collecting human data and establishing human baseline performance under teleoperation and scene-interactive regimes. Our extensive evaluation of state-of-the-art vision-language-action (VLA) and world-action-model (WAM) policies on **RoboDyna** reveals that prediction grants the WAM the ability to anticipate the consequences of its own actions in initially static scenes, but under multi-object motion and complex patterns it fails in both prediction and action. Comparing the synchronized and asynchronous protocols shows that inference latency changes which rollouts succeed more than how many, with systematic losses on time-critical tasks. Further evaluation shows that policies generalize to unseen compositions of seen dynamics. The code, benchmark, and complete results are available at <https://robodyna.github.io/robodyna/>.

## 1 Introduction

Physical competence has two components: knowing how the scene will evolve, and how to respond to it. Current benchmarks score neither. Predictive world model benchmarks NVIDIA et al. [2025], Liao et al. [2025] emphasize physical reasoning while omitting measuring action competencies. Robotics benchmarks, on the other hand, either lack dynamic elements Gu et al. [2023], Liu et al.

[2023], Nasiriany et al. [2024] or simulate simplified physically implausible dynamics where the world halts while the policy infers Fang et al. [2026]. Evaluating under the latter regime has several consequences. First, **the measured physical competency does not transfer to the real-world**. Second, **physical anticipation is never necessary**—If the world waits, acting on the current observation is always correct. Third, **inference cost is free**—A policy taking seconds per chunk Feng et al. [2025] scores like one running at tens of Hz Black et al. [2024].

Designing a comprehensive benchmark that measures action competency in dynamic world is essential, in particular for world-action models (WAMs), which are designed to address this problem: a policy that imagines physical evolution should act on where things *will be* Liao et al. [2025], Kim et al. [2026], Zheng et al. [2026]. Today that proposition is assessed by generative proxies, such as video quality, or by task success in a static, synchronized world (Section 2). Neither tests whether imagination improves control under time pressure.

To this end, we propose **RoboDyna**, a physical, dynamic, *latency-aware benchmark* for dual-arm manipulation (see Figure 1). **RoboDyna** makes physical prediction necessary, charges its cost, and lets any advantage register as task success. The proposed benchmark consists of *20 conceptual tasks*, divided into five physical understanding categories with varying difficulty level to measure core competencies of the policies; *10 complex household tasks* to evaluate policies under real-world conditions; *Mixed metrics* to record full and partial success under *synchronous and asynchronous* regimes; 3500 expert demonstrations and interactive tools to collect human data and establish baseline performance under two controls, teleoperation and scene-interactive.

In summary, the contributions of our work are as follows: a novel benchmark suite of **physics-governed dynamic tasks** in five capability categories with composable difficulty factors and **latency-charged asynchronous protocol** evaluation protocols; Extensive evaluation of state-of-the-art VLAs and WAMs to highlight their physical competencies under different conditions; Human baseline performance under different control regimes to measure their agreement with the robotic policies.

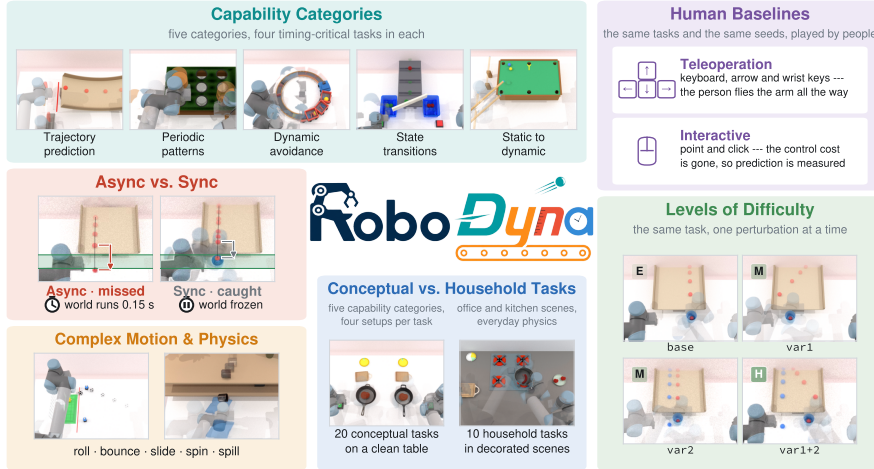


Figure 1: **RoboDyna at a glance**. 20 conceptual tasks in five capability categories and 10 household tasks with physics-governed motion; four difficulty settings per conceptual task; paired sync/async protocols; and a human study on the same seeds.

## 2 What Static, Synchronized Evaluation Cannot Measure

Acting competently in a dynamic world requires prediction and action together. Recent world-action models have shown promise on video prediction benchmarks NVIDIA et al. [2025], Feng et al. [2025] and across almost all robotic benchmarks Liao et al. [2025], Kim et al. [2026], Zheng

et al. [2026], Zhang et al. [2026], yet no existing evaluation measures the two abilities, prediction and action, jointly.

On the prediction side, dynamic world-model benchmarks score predictive ability directly, through video quality, physical plausibility, or forecasting accuracy NVIDIA et al. [2025], Liao et al. [2025], and never require the model to act on its own prediction. World-in-World Zhang et al. [2025] moves toward closed-loop evaluation but scores success and controllability rather than acting under continuous dynamics and inference delay. Whether an accurate prediction converts into a timely action is exactly what these protocols leave untested.

On the action side, manipulation benchmarks lack physics-grounded dynamics. Large-scale suites target generalizable skills Gu et al. [2023], Nasiriany et al. [2024], complex manipulation Liu et al. [2023], and long-horizon tasks Mees et al. [2022], all in scenes that do not change on their own, so there is nothing to anticipate. Emerging benchmarks with object motion Fang et al. [2026] animate the targets of otherwise static tasks with scripted motion. The resulting dynamics are not physically plausible: objects translate sideways or drift randomly through the scene, unconstrained by gravity or friction. In these benchmarks the clock is also synchronized: the scene halts while the model runs inference, so inference time never enters the score and acting on the current observation is always correct. Concurrent work relaxes parts of this convention with asynchronous evaluation Xie et al. [2026], Chen et al. [2026], but none scores the same episode under both a synchronized and an asynchronous setting.

**RoboDyna** evaluates both abilities at once: its tasks are built around physics-governed dynamics in which objects roll, bounce, slide, and spin under gravity and friction (Section 3), it charges each policy its own measured delay and replays identical seeds under both protocols, and it provides a human baseline against which policy performance can be compared.

### 3 The RoboDyna Benchmark

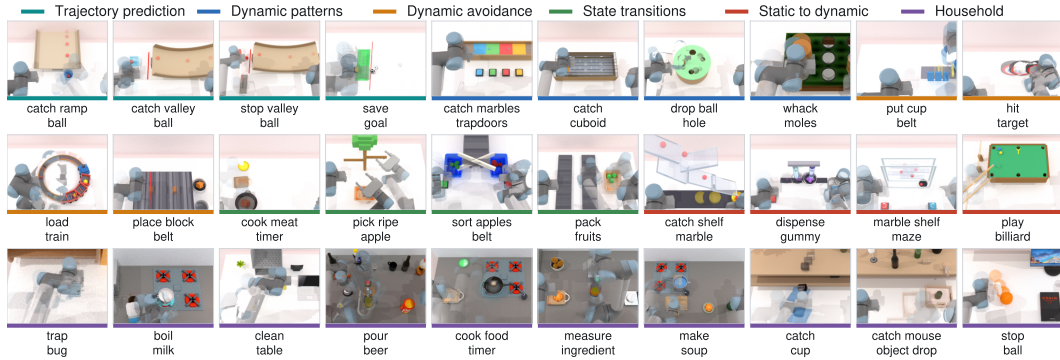


Figure 2: **The 30 RoboDyna tasks.** Rows one and two are the 20 conceptual tasks in category order; row three is the 10 household scenes.

Each **RoboDyna** task is a language-conditioned dual-arm manipulation problem in SAPIEN Xiang et al. [2020] whose scene evolves under its own dynamics. The benchmark consists of 20 *conceptual* tasks and 10 *household* tasks; Figure 2 shows all 30, and the full listing can be found in Appendix A.

**Conceptual tasks.** The conceptual tasks isolate physical understanding in five capability categories, each with four tasks. Objects are simple (cubes, balls, fruits), and the scene contains nothing beyond the assets the task needs. Each category assesses a distinct aspect of that understanding. *Trajectory prediction*: an object moves toward a goal position, either straight or redirected by a bounce. *Periodic-pattern understanding*: the scene contains a periodic motion, such as a revolving platform; the robot must understand and predict the pattern and act at the right moment. *Dynamic avoidance*: the robot interacts with a moving object, such as a train, while avoiding a static or moving obstacle.

Table 1: Success rate by capability category, pooled over both protocols. *E* (easy) is the base condition; *M* (medium) averages var1 and var2. *Overall* covers every task under every condition, including var1+2. Bold marks the best policy per column.

| Policy      | Traj. pred. |             | Periodic pattern |             | Dyn. avoid. |             | State trans. |             | Static to dyn. |             | Conceptual  |             | Household   | Overall     |
|-------------|-------------|-------------|------------------|-------------|-------------|-------------|--------------|-------------|----------------|-------------|-------------|-------------|-------------|-------------|
|             | E           | M           | E                | M           | E           | M           | E            | M           | E              | M           | E           | M           |             |             |
| $\pi_{0.5}$ | <b>0.24</b> | <b>0.20</b> | <b>0.28</b>      | <b>0.14</b> | <b>0.36</b> | <b>0.38</b> | 0.15         | 0.09        | 0.10           | <b>0.09</b> | <b>0.23</b> | <b>0.18</b> | <b>0.07</b> | <b>0.17</b> |
| X-VLA       | 0.23        | 0.17        | 0.06             | 0.02        | 0.24        | 0.11        | <b>0.24</b>  | <b>0.18</b> | 0.05           | 0.03        | 0.16        | 0.10        | <b>0.07</b> | 0.11        |
| FastWAM     | 0.04        | 0.08        | 0.00             | 0.00        | 0.00        | 0.00        | 0.12         | 0.12        | <b>0.17</b>    | 0.07        | 0.07        | 0.06        | 0.04        | 0.05        |

*State-transition estimation*: an object or the environment changes state, such as an apple ripening. *Static-to-dynamic understanding*: nothing moves at the start; the robot must produce the desired motion itself, as in billiards. Each conceptual task comes in four conditions: base (*easy*), var1 and var2, which each switch on one task-specific perturbation axis (*medium*), and their composition var1+2 (*hard*). For instance, in the *catch marbles* task, base offers a single colored marble and up to three catch attempts; var1 lets the trapdoor open only once; var2 adds a black distractor marble that must not be caught; var1+2 composes both.

**Household tasks.** The 10 household tasks move the same demands into everyday office and kitchen scenes, with visually enriched backgrounds and ordinary physical processes in place of contrived mechanisms: milk rises and overflows, a coffee spill spreads toward a laptop, or beer foams toward the rim. Each task requires one or more physical skills. In *pour beer*, for instance, the robot must read the rising foam and pause pouring to avoid overflow; in *catch cup* it must catch a falling fragile object on a soft surface.

**Metrics and evaluation protocol.** Alongside binary success, we report a *progress score*  $PS = b \cdot 2^{-m} \in [0, 1]$ , where  $b$  credits observable completed milestones and  $m$  counts penalty events, so zero-success regimes stay measurable; the milestones and penalties are task-specific (in *pour beer*, fill-level bands earn 0.25–0.75 and an overflow earns nothing). We group the four conditions into three difficulty levels: base is *easy*, var1 and var2 are *medium*, and var1+2 is *hard*, since it is the combination left out of the training demonstrations. We provide 50 expert demonstrations for each of the 70 configurations excluding var1+2, which is deliberately left undemonstrated so it acts as a test of compositional generalization. All three policies are fine-tuned on the full set of 3,500 demonstrations and evaluated from a single checkpoint each ( $\pi_{0.5}$  and X-VLA at 30k gradient steps, FastWAM at 13k). Every task-condition-protocol slot runs the same ten held-out seeds, giving 900 episodes per policy per protocol and 5,400 in total. The simulator steps physics at 250 Hz and holds one policy action for 15 steps (60 ms, the demonstration cadence); each policy commits its official chunk length before observing again. Under *sync* the world is frozen while the policy computes. Under *async* the wall-clock time of each forward pass is measured and charged to simulated time before the chunk lands; during the charge the arm holds its last commanded joint target while the scene keeps evolving. A single charge is capped at 1 s, and the first call of each process is discarded so that one-off graph compilation is not billed. The mean charge is 109 ms for  $\pi_{0.5}$ , 156 ms for X-VLA and 236 ms for FastWAM.

## 4 Experiments

We evaluate two state-of-the-art VLAs,  $\pi_{0.5}$  Physical Intelligence et al. [2025] and X-VLA Zheng et al. [2025], and one WAM, FastWAM Yuan et al. [2026], which rolls a video prediction forward and decodes an action chunk from it. With a single WAM evaluated, WAM-level statements below are observations about this policy rather than the class. We answer three research questions: does the WAM’s prediction give it an advantage over the VLAs (**RQ1**)? How much does inference latency matter (**RQ2**)? And do policies generalize to unseen dynamic compositions (**RQ3**)? Paragraphs R1–R3 below answer them in turn.

Table 2: Success rate and mean progress score (SR/PS) by capability category and protocol. Conceptual cells average the easy and medium conditions; Avg. is the mean over the five categories. The mean charged latency is shown beside each policy.

| Policy               | Protocol | Traj.<br>pred. | Periodic<br>pattern | Dyn.<br>avoid. | State<br>trans. | Static to<br>dyn. | Household | Avg.      |
|----------------------|----------|----------------|---------------------|----------------|-----------------|-------------------|-----------|-----------|
| $\pi_{0.5}$ (109 ms) | sync     | 0.23/0.27      | 0.24/0.28           | 0.39/0.47      | 0.16/0.30       | 0.10/0.20         | 0.06/0.21 | 0.22/0.30 |
|                      | async    | 0.21/0.25      | 0.18/0.25           | 0.36/0.44      | 0.09/0.26       | 0.09/0.20         | 0.07/0.22 | 0.19/0.28 |
| X-VLA (156 ms)       | sync     | 0.17/0.19      | 0.05/0.08           | 0.14/0.16      | 0.21/0.28       | 0.04/0.11         | 0.07/0.13 | 0.12/0.16 |
|                      | async    | 0.23/0.27      | 0.03/0.07           | 0.21/0.23      | 0.21/0.29       | 0.04/0.10         | 0.06/0.14 | 0.14/0.19 |
| FastWAM (236 ms)     | sync     | 0.06/0.08      | 0.00/0.04           | 0.00/0.00      | 0.19/0.31       | 0.12/0.16         | 0.02/0.08 | 0.07/0.12 |
|                      | async    | 0.06/0.08      | 0.00/0.03           | 0.00/0.01      | 0.06/0.25       | 0.12/0.15         | 0.05/0.15 | 0.05/0.11 |

**R1: the WAM’s prediction gives it capabilities disjoint from the VLAs’.** FastWAM ranks last overall although it explicitly predicts the future (0.05 against 0.17 and 0.11, Table 1). Yet it holds the highest success rate on static-to-dynamic understanding at easy (0.17 against 0.10 and 0.05), which is the weakest category for both VLAs. To investigate this, we replayed FastWAM on all 30 tasks and compared its predicted video against the ground truth. Starting from a static frame, FastWAM predicts the robot’s own motion and its consequences, so it can reason about the future of a still scene, which the VLAs cannot. When the scene itself is dynamic, the predictions degenerate: across the 30 tasks the predicted video shows the scene’s own motion on only 12, and on the other 18, it reproduces a near-frozen input or erases the moving object. On all four periodic-pattern tasks the prediction is incorrect or inconsistent, and as a result FastWAM’s periodic-pattern and dynamic-avoidance achieves zero success as shown in Table 1. Conversely, FastWAM never scores above zero on a task where its prediction is actively wrong: an incorrect imagined future always co-occurs with failure, and once the dynamics exceed what the video branch can predict, the task is lost. The VLAs, which never predict, succeed exactly where FastWAM fails: in the rollout videos they simply track the moving object or wait at a location where the task can be completed, succeeding through reaction and trial and error with no account of the future at all. This carries  $\pi_{0.5}$  to 0.24 or more at easy on trajectory prediction, periodic pattern, and dynamic avoidance, but an approach that never predicts the future has an upper bound.

**R2: latency determines which rollouts succeed, not how many.** Table 2 shows moving from synchronous to asynchronous,  $\pi_{0.5}$  and FastWAM only drop 0.03 and 0.02 of average success rate, and X-VLA even gains 0.02. The averages move little, but the successes underneath are not the same successes: pairing protocols seed by seed, a third to a half of every policy’s successes are protocol-contingent ( $\pi_{0.5}$  loses 0.38 of its sync successes and gains 49 new ones). This matches our observation in R1: success rests on accidental timing alignment, and latency re-deals it. Looking into the categories, state transition suffers the largest async losses:  $-0.07$  for  $\pi_{0.5}$  and  $-0.13$  for FastWAM, with X-VLA flat (Table 2); FastWAM, which carries the largest charge, loses the most. A state-transition task evolves in one direction and runs a deadline the robot did not start and cannot pause—the apple ripens and does not turn green again—so the seconds charged during inference are subtracted directly from the window to act in. The largest single effect in the sweep is FastWAM’s on `sort_apples_belt`, a task where the model must sort each apple before the belt carries it off the end: 0.65 sync against 0.17 async. For FastWAM especially, the delay between when an action is predicted and when it is applied must be accounted for; otherwise the mismatch causes a significant drop in success rate.

**R3: action policies can generalize to combined dynamic variations.** All models degrade on the hard condition (`var1+2`), which is unseen during training, but the drop at the composition is gentle (Figure 3): success runs 0.23/0.18/0.14 ( $\pi_{0.5}$ ), 0.16/0.10/0.09 (X-VLA) and 0.07/0.06/0.04 (FastWAM) across easy, medium and hard, retaining 0.56–0.61 of each policy’s easy rate, and for both VLAs the medium→hard step is no larger than the easy→medium step. The collapse concentrates in *periodic pattern*, the only category in Figure 3 where every scoring policy degrades

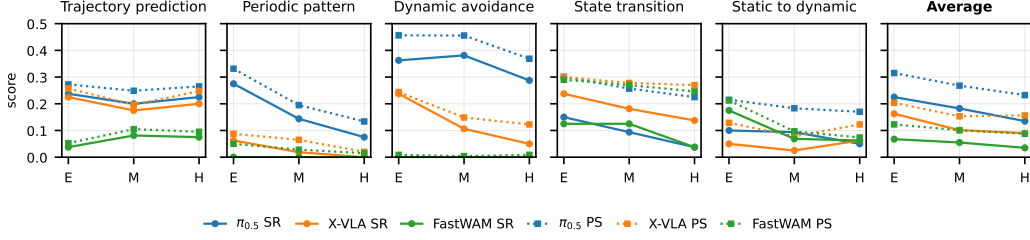


Figure 3: Success rate (solid) and progress score (dotted) across the difficulty ladder, by capability category and pooled over both protocols. *E* is base, *M* averages var1 and var2, and *H* is var1+2, the combination absent from training. *Average* pools the five categories.

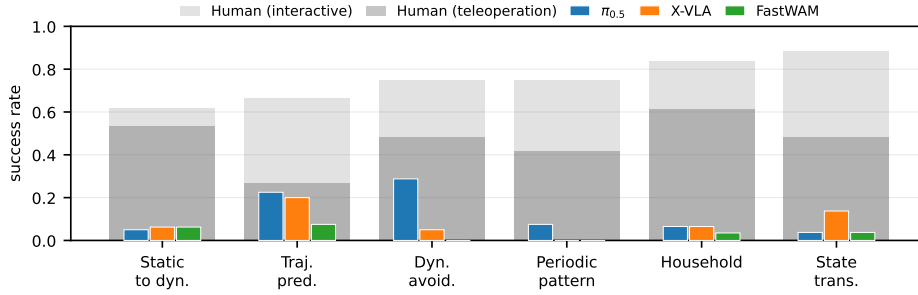


Figure 4: Success rate by category on the human-evaluated slice (conceptual tasks at var1+2, household tasks at base). The shaded backdrop bars give the human level under interactive control (light) and teleoperation (dark); the coloured bars are the policies.

monotonically. Looking at the implementation of the variations, most periodic-pattern variations add an extra object the policy must not lose the pattern to (a distractor marble, a decoy hole, a distractor rabbit) or remove an affordance, such as a retry or a clear view (Table 4), which the results reveal as difficult. On the other hand, *trajectory prediction* is the least impacted, and some cells even improve. Looking at the tasks, three of the four tasks in this type share an added bounce off a wall as one variation axis and a distractor ball as the other, while the ball moves on a surface. When a dynamic phenomenon appears commonly in the training data, it is learned by the model and can be understood in unseen, combined tasks. This further supports that a combination of dynamics is not difficult, but an unseen dynamic can be catastrophic.

**Human performance.** Figure 4 compares the twelve participants’ performance with the policies at the unseen var1+2 condition. Under teleoperation humans solve 50.2% where the best policy solves 14% at the same condition; under interactive control, which removes the cost of acting through the arms, they solve 77.1%. The cost of acting, like model inference, introduces a delay. As a consequence, the gap between the two regimes is largest on the timing-critical categories (trajectory prediction falls from 0.67 to 0.27 and periodic pattern from 0.75 to 0.42) and smallest on static-to-dynamic understanding (0.62 to 0.53).

The human baseline also provides a reference for the difficulty of the tasks themselves. Under interactive control, with the control burden removed, the hardest categories are static-to-dynamic understanding and trajectory prediction, where the prediction itself is naturally harder (a ball moving on a ramp, marbles moving through a maze), while everyday state transitions (0.88) and household tasks (0.84) are easiest. The VLAs are relatively strongest where humans struggle most: on this slice  $\pi_{0.5}$  peaks on dynamic avoidance (0.29) and X-VLA on trajectory prediction (0.20, where  $\pi_{0.5}$  scores higher still at 0.23), categories where tracking a visible object and reacting suffices;

both collapse on periodic pattern (0.07 and 0.00), which humans handle at 0.75. The per-category values quoted here are the ones plotted in Figure 4. The VLAs behave like a mapping from the current observation to an action, while humans reason about the dynamics and stay strong even on complicated periodic patterns. The WAM suffers most in reasoning about complex dynamics, as R1 discussed, reaching zero on dynamic avoidance and periodic pattern.

## 5 Conclusion

We proposed **RoboDyna**, a benchmark that evaluates physical prediction and action jointly: physics-governed dynamic tasks in five capability categories with composable difficulty factors, paired synchronized and asynchronous protocols that charge each policy its own measured latency on identical seeds, a progress score for near-zero regimes, and a human baseline under two control regimes. Evaluating state-of-the-art VLAs and a WAM, we showed that prediction buys a different competence rather than a better one: the WAM leads the one category where scenes start still, yet ranks last overall at 0.05 success, because its imagined futures degenerate once the world moves on its own. We showed that charging latency moves average success by at most 0.03 yet re-deals a third to a half of every policy’s successes, with systematic losses exactly where the world owns the clock. We further showed that policies retain 0.56–0.61 of their success on compositions of dynamics left out of training—what breaks is an unseen dynamic, not an unseen combination—and that every policy remains far below humans, who solve 50% under teleoperation where the best policy solves 14%.

These findings turn whether prediction buys control into a measurable agenda: for prediction to do so, it must attend to the objects that move, arrive within the task’s action window, and outrun its own inference cost. **RoboDyna** measures all three.

## References

- Kevin Black, Noah Brown, Danny Driess, et al.  $\pi_0$ : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Yuxuan Chen, Wanruo Zhang, and Xiao Li. Reflex: Enabling fast and predictive vision-language-action models for reaction-critical manipulation. *arXiv preprint arXiv:2608.14379*, 2026.
- Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkan Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*, 2026.
- Yao Feng, Chendong Xiang, Xinyi Mao, Hengkai Tan, Zuyue Zhang, Shuhe Huang, Kaiwen Zheng, Haitian Liu, Hang Su, and Jun Zhu. Vidarc: Embodied video diffusion model for closed-loop control. *arXiv preprint arXiv:2512.17661*, 2025.
- Jiayuan Gu, Fanbo Xiang, Xuanlin Li, et al. ManiSkill2: A unified benchmark for generalizable manipulation skills. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2302.04659.
- Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos Policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026.
- Yue Liao, Pengfei Zhou, Siyuan Huang, others, Maoqing Yao, and Guanghui Ren. Genie Envisioner: A unified world foundation platform for robotic manipulation. *arXiv preprint arXiv:2508.05635*, 2025.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters (RA-L)*, 2022. [arXiv:2112.03227](#).
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.
- NVIDIA, Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, et al. Cosmos world foundation model platform for physical AI. *arXiv preprint arXiv:2501.03575*, 2025.
- Physical Intelligence, Kevin Black, Noah Brown, Chelsea Finn, Sergey Levine, et al.  $\pi_{0.5}$ : A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X. Chang, Leonidas J. Guibas, and Hao Su. SAPIEN: A simulated part-based interactive environment. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhong Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026.
- Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- Jiahao Zhang, Muqing Jiang, Nanru Dai, others, Yilun Du, and Jieneng Chen. World-in-World: World models in a closed-loop world. *arXiv preprint arXiv:2510.18135*, 2025.
- Zhanguang Zhang, Zhiyuan Li, Behnam Rahmati, Rui Heng Yang, Yintao Ma, Amir Rasouli, Sajjad Pakdamansavoji, Yangzheng Wu, Lingfeng Zhang, Tongtong Cao, Feng Wen, Xinyu Wang, Xingyue Quan, and Yingxue Zhang. Do world action models generalize better than VLAs? a robustness study. *arXiv preprint arXiv:2603.22078*, 2026.
- Jia Zheng, Teli Ma, Yudong Fan, Zifan Wang, Shuo Yang, and Junwei Liang. MotionWAM: Towards foundation world action models for real-time humanoid loco-manipulation. *arXiv preprint arXiv:2606.09215*, 2026.
- Jinliang Zheng, Jianxiong Li, Zhihao Wang, others, Tai Wang, and Xianyu Zhan. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.

## A Task list

Table 3 lists the 30 tasks, and Table 4 the two perturbation axes of each conceptual task; var1 switches on the first, var2 the second, and var1+2 both. Full per-task, per-condition, per-protocol results will be published with the benchmark upon acceptance.

Table 3: The 20 conceptual tasks in five capability categories, plus 10 household tasks.

| Category              | Tasks                                                                                                                                      |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Trajectory prediction | catch_ramp_ball, catch_valley_ball, stop_valley_ball, save_goal                                                                            |
| Periodic pattern      | catch_marbles_trapdoors, catch_cuboid, drop_ball_hole, whack_moles                                                                         |
| Dynamic avoidance     | put_cup_belt, hit_target, load_train, place_block_belt                                                                                     |
| State transition      | cook_meat_timer, pick_ripe_apple, sort_apples_belt, pack_fruits                                                                            |
| Static to dynamic     | catch_shelf_marble, dispense_gummy, marble_shelf_maze, play_billiard                                                                       |
| Household             | boil_milk, catch_cup, catch_mouse_object_drop, clean_table, cook_food_timer, make_soup, measure_ingredient, pour_beer, stop_ball, trap_bug |

Table 4: The two perturbation axes of each conceptual task. var1 switches on the first, var2 the second, and var1+2 both. The axes are task-specific: the numbering identifies which switch is thrown, not a kind of perturbation shared across tasks, which is why we report the difficulty result at the level of *how many* axes are active rather than which one. Some axes change the physics, some add a distractor, some remove an affordance such as a retry or a clear view.

| Task                    | var1                             | var2                             |
|-------------------------|----------------------------------|----------------------------------|
| catch_ramp_ball         | ball bounces off a wall first    | a distractor ball                |
| catch_valley_ball       | ball bounces off a wall first    | a distractor ball                |
| stop_valley_ball        | ball bounces off a wall first    | a distractor ball                |
| save_goal               | opposing players on the field    | the goal is partly covered       |
| catch_marbles_trapdoors | the door opens once, no retry    | a black marble not to be caught  |
| catch_cuboid            | two cuboids to catch             | opaque surface hides the landing |
| drop_ball_hole          | the ball sticks to the surface   | a decoy hole                     |
| whack_moles             | a distractor mole                | the moles relocate between pops  |
| put_cup_belt            | curtains occlude the belt        | the curtains move                |
| hit_target              | a blocker in front of the target | the blocker moves                |
| load_train              | one specific wagon is the target | the train runs through a tunnel  |
| place_block_belt        | the receiving bowl moves         | a blocker                        |
| cook_meat_timer         | a cook button must be pressed    | two cooking setups               |
| pick_ripe_apple         | two apples                       | the basket moves                 |
| sort_apples_belt        | colours arrive in random order   | 0.3 of the apples are rotten     |
| pack_fruits             | two fruit colours                | a distractor                     |
| catch_shelf_marble      | the marble reacts and rolls      | the shelf oscillates             |
| dispense_gummy          | randomised layout                | the belt never stops             |
| marble_shelf_maze       | the ball keeps moving            | the receiving bowl oscillates    |
| play_billiard           | one specific pocket is required  | distractor balls                 |